

PO-LICE - Unified Master Document

Purchase Order Liability, Intelligence & Compliance Engine

Kaya AI IIT India Hackathon 2026 | Track 3: Procurement

1. Project Identity, Title & Core Claims

Project Name: PO-lice (Purchase Order Liability, Intelligence & Compliance Engine)

Presenter Details: Team TensorTruss, IIT Madras

Tagline: Catching bid bandits before they break your budget.

What PO-lice Is:

- Amber's procurement enforcement layer - a hard-gate validation module that extends Kaya's AI agent into the procurement decision point before PO approval.
- An audit layer that sits between vendor bid receipt and PO approval.
- A system where LLMs extract and explain; SQL/math decides.

What PO-lice Is Not:

- A system that lets an LLM vote on engineering compliance.
- A replacement for human procurement officers.
- A dismissal of existing carbon tools (EC3, enterprise carbon accounting, etc.) - PO-lice differentiates by combining bid-level technical, vendor, carbon, schedule, and TCO² validation into one hard gate.

Defensible Originality: The Evidence Board - PO-lice does not merely say a bid failed; it maps exactly how one substituted line item creates engineering, vendor, carbon, schedule, and cost consequences downstream.

2. Market Need, Opportunity & Problem Statement

The Procurement Blind Spot

Construction procurement still over-weights upfront price and promised delivery date. Hidden substitutions break the project later.

What Teams See	What Is Hidden	What Breaks Later
Price looks lower	Power draw +10%	Rework and redesign
Delivery date looks faster	Floor load limit breach	Idle crews and schedule slips
PDF bid looks "equivalent"	Missing certificate	Carbon budget breach
	Higher water usage	Higher 5-year OPEX

The Human & Financial Cost

- 10,000+ purchase orders per large infrastructure project - a scale that defies manual audit.
- Procurement inefficiencies and late-discovered substitutions routinely drive cost overruns of 10–20% on complex infrastructure projects, with mega-projects often experiencing far worse leakage.
- A global shortage of specialized MEP engineers leaves the people who could catch this stretched too thin to manually audit hundreds of pages per bid. The sector faces a gap of 650,000 skilled workers in construction-related energy trades, with wage escalation of 20% since 2020 squeezing margins and forcing firms to decline projects despite strong pipelines.
- Procurement decisions are made across disconnected systems: engineering constraints live in BIM databases, vendor history sits in old emails/contracts, sustainability teams track carbon in separate tools - but the PO is often approved before anyone connects the consequences.

The Hook: It is rarely the most expensive bid that is actually the safest one.

Market Opportunity

The global construction management software market is projected to exceed \$5 billion by 2028, growing at ~10% CAGR. Within this, procurement and bid-management tools represent the largest unmet need for deterministic validation. PO-lice targets Tier-1 EPC contractors, mission-critical infrastructure owners (data centers, hospitals, metros), and government PWDs managing multi-crore projects where a single substitution can cascade into multi-crore losses. As a Kaya platform module, PO-lice extends Amber's existing project intelligence into the procurement hard gate.

3. Value Proposition & Solution: The Four Patrols

Core Value: PO-lice transforms procurement from a paperwork exercise into a decision-grade validation layer by reading unstructured vendor bids, extracting structured specifications, and running them through deterministic hard gates - before the PO is signed.

The Core Move: LLM Extracts, Math Decides

Use AI where ambiguity exists. Use deterministic logic where consequences are irreversible.

Layer	Role	Output
LLM Layer	Reads messy PDFs, tables, specs, certificates, and contract clauses.	Structured bid JSON + explanation
SQL / Math Layer	Checks hard constraints: power, weight, water, carbon, dates, certificates.	PASS / FAIL + exact reason
Human Layer	Approves, rejects, or escalates with evidence.	Signed PO or returned bid

Architecture: Vendor PDF → The Docket

Vendor PDF → Document Intelligence → Pydantic JSON → The Squad → The Docket

The critical insight: PO-lice does not ask the LLM "Is this compliant?" It asks SQL/math to prove it.

The Four Patrols

Every patrol is deterministic. No generative model gets a vote on the outcome.

Patrol	Question	Method	Output
Patrol 1 - Building Patrol (Engineering Compliance)	Does the equipment violate BIM/site limits?	SQL matcher against Amber's Project Graph (PostgreSQL constraints)	PASS / FAIL
Patrol 2 - Green Patrol (Carbon Budget)	Does this bid exceed allocated embodied carbon?	EPD lookup + SQL against carbon-budget table	PASS / FAIL
Patrol 3 - Vice Squad (Vendor Reliability)	Has this vendor created hidden delay/cost before?	RAG memory over historical contracts, invoices, delays	Risk Score 1–10
Patrol 4 - Traffic Control (Schedule Ripple)	If this is late or non-compliant, what downstream task breaks?	Rules + graph + probabilistic time-series (Bayesian Monte Carlo)	Delay exposure (days/weeks)

Patrol 1 - Building Patrol Example (Energy Balance):

Dynamic cooling-load limits are checked with a schematic energy-balance inequality. For a chilled-water system, the total imposed load must not exceed the plant capacity:

$$\dot{Q}_{load} = \sum_{i=1}^n \dot{m}_i c_{p,i} (T_{out,i} - T_{in,i}) + \dot{Q}_{parasitic} \leq \dot{Q}_{plant,max}$$

If the vendor's equipment pushes past the cooling cap $\dot{Q}_{plant,max}$, the system triggers a hard mechanical reject - no LLM opinion involved.

Patrol 3 - Vice Squad (Agentic RAG):

If a required document (e.g., an OSHA-style safety certificate or insurance certificate) is missing, the agent does not just flag it: it retrieves the document from the enterprise database or generates a draft email and hands off to Jarvis for autonomous vendor outreach. In the Stage 1 prototype, this workflow is demonstrated end-to-end; full Jarvis handoff requires Kaya platform integration in Stage 2.

Patrol 4 - Traffic Control (Schedule Ripple):

Delay risk signals are fed back into MCP Planner™ for dynamic schedule re-optimization. PO-lice doesn't just flag; it closes the loop with Kaya's existing planning engine.

4. Underlying Magic

The Secret Sauce: PO-lice is not a model. It is a boundary architecture - a hard firewall between generative cognition and deterministic judgment.

Most AI procurement tools let the LLM "guess" at compliance by pattern-matching historical bids. PO-lice does the opposite:

- 1. **Cognitive Extraction:** Open-source LLMs and document-intelligence models read spatial layouts, tables, and nested clauses, outputting strictly validated Pydantic JSON. If a field is malformed, the pipeline fails loudly - it does not hallucinate a value.
- 2. **Deterministic Judgment:** Every patrol runs SQL/math against ground-truth constraint tables drawn from Amber's Project Graph. The LLM never sees the pass/fail decision.
- 3. **The Evidence Board:** Instead of a boolean reject, PO-lice outputs a directed consequence graph showing exactly which engineering limit was breached, which carbon budget was blown, and which schedule milestone will slip - all traceable to a specific cell in the source PDF.

This boundary is what makes PO-lice enterprise-credible. CTOs do not trust black-box AI with thermodynamic limits. They trust PostgreSQL tables, EPD lookups, and deterministic inequalities. PO-lice gives them exactly that, wrapped in a command-center UI that makes the evidence actionable.

5. The Evidence Board (Unique Differentiator)

Level 1 differentiator: One substituted line item becomes a visible map of engineering, carbon, vendor, schedule, and cost consequences.

Visual Concept: A directed graph starting from a single node:

- **Root:** "Vendor substitutes chiller model - looks equivalent in the bid PDF"
- **Branches:**
 - Power draw +10% → Electrical panel redesign → Reject or escalate
 - Water usage +15% → Sustainability + cooling risk → Carbon/OPEX penalty
 - Floor load +8% → Structural tolerance breach → Hard fail
 - Vendor late 3/5 times → ROJ window risk → Schedule contingency
 - Missing certificate → Compliance hold → Legal/procurement flag

PO-lice does not just detect a mismatch. It explains what the mismatch will break downstream.

6. The Docket & TCO² Insight

TCO² Defined: Total Cost of Ownership squared - upfront capex + 5-year OPEX + carbon liability + schedule-risk-adjusted cost.

The Aha Moment: Vendor B looks cheapest upfront, but PO-lice proves it is the most expensive and riskiest once 5-year operating cost, carbon liability, vendor reliability, and schedule exposure are included.

Vendor	Upfront Cost	Engineering	Vendor Risk	Carbon	Schedule Risk	5-Year TCO²	Decision
Vendor A	₹4.2Cr	PASS	Low	PASS	Low	₹6.0Cr	Recommended
Vendor B	₹3.8Cr	FAIL	High	FAIL	Medium	₹6.8Cr	Reject
Vendor C	₹4.5Cr	PASS	Medium	PASS	Low	₹6.0Cr	Acceptable

Unified Output: Technical status + vendor risk + carbon status + schedule exposure + 5-year TCO² ranking.

7. Competitive Analysis

Conservative, defensible claim: Existing tools are not dismissed; PO-lice differentiates through combined bid-stage hard-gate validation.

Real-World Competitive Landscape

Capability	Autodesk BuildingConnected + TradeTapp	Procore	EC3 (Building Transparency)	CO2 AI / Persefoni	SAP Ariba / Coupa	SmartBid (ConstructConnect)	PO-lice
Bid comparison & RFP mgmt	✓	✓	✗	✗	✓	✓	✓
PDF/spec extraction	partial	partial	✗	✗	partial	partial	✓
Engineering/BIM compliance hard gate	✗	✗	✗	✗	✗	✗	✓
Vendor reliability performance RAG	partial (qualification)	✗	✗	✗	✗	✗	✓ (delivery history)
Embodied carbon calculation	✗	✗	✓ (material/project)	✓ (corporate)	✗	✗	✓ (equipment-level bid gate)
Carbon as hard bid gate	✗	✗	partial	✗	✗	✗	✓
Schedule ripple / downstream impact	✗	✗	✗	✗	✗	✗	✓
TCO ² : cost + OPEX + carbon + risk	✗	✗	✗	✗	✗	✗	✓

Competitor Deep-Dive:

- **Autodesk Construction Cloud / BuildingConnected:** The market leader in bid management and subcontractor qualification. TradeTapp performs financial risk scoring and insurance verification. However, it does not perform equipment-level engineering compliance (e.g., cooling-load inequalities) nor carbon hard-gating at the PO stage.
- **Procore:** Dominant in project management and cost tracking. Its procurement module handles RFIs and change orders, but lacks deterministic validation against BIM constraints or embodied carbon budgets.
- **EC3 (Building Transparency):** The gold standard for embodied carbon benchmarking. It supports procurement-phase EPD lookups and carbon comparisons. However, it does not integrate vendor reliability, schedule risk, or engineering physics into a single pass/fail gate.
- **CO2 AI / Persefoni / Watershed:** Enterprise carbon-accounting platforms focused on Scope 1/2/3 corporate reporting and regulatory disclosure (CSRD, SEC). They track emissions after procurement decisions are made; they do not gate individual purchase orders.
- **SAP Ariba / Coupa:** Generic enterprise procurement suites with strong workflow and spend analytics. They are not construction-native and have no concept of MEP engineering limits, EPD lookups, or schedule ripple.
- **SmartBid (ConstructConnect):** Document distribution and bid invitation platform. No deterministic validation layer.

The Gap: Bid-management tools validate paperwork. Carbon tools calculate emissions at the project or corporate level. No existing tool gates a single purchase order on engineering physics, vendor track record, and carbon budget simultaneously before the money moves. That gap is PO-lice.

8. Customer Segments

Primary Segments:

1. **Tier-1 EPC Contractors:** Managing ₹500Cr+ infrastructure projects with 10,000+ POs. Pain point: manual bid review cannot scale.
2. **Mission-Critical Asset Owners:** Data center operators, hospital chains, metro rail authorities. Pain point: substitution-induced downtime is catastrophic.

- 3. **Government PWDs & Smart City Agencies:** Mandated carbon budgets and public audit trails. Pain point: need deterministic, evidence-based compliance.
- 4. **Sustainability-First Developers:** LEED / GRIHA-certified projects where embodied carbon is a hard constraint. Pain point: carbon tools are disconnected from procurement.

Early Adopters: IIT research incubators, mid-size MEP contractors with 50+ POs/month, and Kaya/Amber pilot projects.

9. Kaya Platform Value & Integration Economics

Integration Model: PO-lice is designed as a native module within the Kaya platform, not a standalone product. It leverages Kaya's existing project data, user base, and distribution to deliver procurement-stage validation without requiring customers to adopt a separate tool.

Value Streams for Kaya:

Value Stream	Description	Impact
Platform Retention	Procurement teams stay inside Kaya for bid validation instead of exporting to spreadsheets	Reduced churn, higher stickiness
Upsell Path	PO-lice becomes a premium module for enterprise Kaya accounts	Increased ARPU per project
Data Network Effects	Every bid processed improves the Vice Squad RAG corpus for all Kaya users	Competitive moat
Compliance Lock-in	Public-sector clients require deterministic audit trails that PO-lice generates	Mandatory for government tenders

Resource Economics (Target Year 3 post-integration):

- **Development Cost:** ₹20–30L one-time build + ₹8L/year maintenance
- **Value per Prevented Substitution:** ₹2–5Cr saved per incident on data center / hospital projects
- **Break-even:** 1 prevented substitution per 100 enterprise projects
- **Margin Structure:** 85%+ incremental margin for Kaya (software module, minimal per-bid marginal cost)

Viability: PO-lice is viable because every prevented substitution pays for the module. A single caught chiller substitution on a data center project can save ₹2–5Cr in rework and delay. At the cost of a Kaya premium module, the ROI is demonstrable in the first prevented incident.

10. Kaya Rollout Strategy, Channels & Customer Relationships

Rollout Strategy within Kaya's Ecosystem:

Phase	Timeline	Tactic
Phase 1: Hackathon → Pilot	Months 1–3	IIT hackathon prototype → Kaya/Amber internal pilot → 3 lighthouse customer projects at zero incremental cost
Phase 2: Module Beta	Months 4–9	Soft launch as "Kaya Procurement Intelligence" beta for enterprise accounts; conference demos at CII / NICMAR
Phase 3: Platform Native	Months 10–18	Full native integration into Kaya's project dashboard; default module for enterprise tier

Channels:

- **Kaya Platform:** Embedded module inside existing Kaya construction intelligence platform.
- **Kaya Sales Team:** Enterprise outreach to procurement heads and sustainability officers as part of existing Kaya sales motion.
- **System Integrator Channel:** MEP consultants and BIM studios already using Kaya resell advanced procurement validation as part of their advisory stack.
- **Academic Channel:** IIT incubators provide credibility and pilot sandboxes.

Customer Relationships:

- **Acquisition:** Free 30-day pilot on 50 synthetic bids to prove value.
- **Retention:** Quarterly "Substitution Prevented" reports showing TCO² savings.

- **Growth:** Land-and-expand - start with one project, expand to entire portfolio.

11. Key Partnerships

Partner Type	Target Partners	Value Exchange
Platform Anchor	Kaya AI / Amber	Distribution, data pipeline, credibility
BIM / CAD	Autodesk, Bentley Systems	Connector APIs, native file format support
Carbon Data	Building Transparency (EC3), EPD databases	Verified carbon factors, regulatory alignment
ERP / Procurement	SAP, Oracle, Procore	PO system integration, workflow embedding
Industry Bodies	CII, NICMAR, ISHRAE	Standards alignment, certification, advocacy
Open-Source Infrastructure	Ollama, Hugging Face, Meta AI	Local LLM inference, embedding models, community support

12. Development Resources, Key Activities & Cost Structure

Key Resources:

- **Intellectual:** Pydantic schemas, SQL constraint templates, synthetic PDF corpus, vendor-history RAG corpus.
- **Technical:** Self-hosted PostgreSQL, Ollama inference server, MinIO object storage, Redis cache, Next.js hosting.
- **Financial:** Hackathon seed support + Kaya integration budget for 18-month development cycle.

Key Activities:

1. Extraction Engine Tuning: Improving PDF-to-JSON accuracy for construction specs using open-source document parsers.
2. Constraint Table Curation: Building BIM-derived SQL baselines for each asset class from Amber's Project Graph.
3. RAG Corpus Expansion: Ingesting vendor contracts, delivery logs, and dispute records.
4. Dashboard & API Development: The Precinct UI and Kaya-native REST API.
5. Pilot Validation: Running live bid audits with lighthouse customers.

Cost Structure (Monthly, Year 1):

Expense	Amount
Cloud / Self-Hosted Infrastructure (GPU inference, PostgreSQL, Redis, MinIO)	₹1.0L
Engineering (2 full-stack, 1 ML)	₹3.0L
Kaya Integration Support	₹0.5L
Legal / Compliance	₹0.3L
Total Monthly Development Cost	₹4.8L
Year 1 Resource Requirement	₹58L

13. Feature Development Team

Name	IIT Affiliation
Aparna Jha	IIT Madras
Jb Anmol	IIT Madras
Pratham Amritkar	IIT Madras

Advisory Support: [Kaya product executive, former CTO of Tier-1 EPC, sustainability lead from data center operator - to be recruited for integration guidance.]

14. Kaya Platform Impact Metrics

Three-Year Integration Forecast:

Metric	Year 1	Year 2	Year 3
Kaya Projects Using PO-lice	3 pilots	12 enterprise	35 enterprise
Bids Audited via Module	2,500	25,000	1,00,000
Engineering Clashes Caught	45	420	1,800
Carbon Budget Breaches Prevented	12	110	480
Module Team Size	4	8	15
Kaya Platform Gross Margin Impact	N/A	+75% module margin	+82% module margin

Key Performance Indicators (KPIs):

- **False Positive Rate:** <5% (target: deterministic gates should not reject valid bids)
- **Extraction Accuracy:** >92% F1-score on structured field recovery from PDFs
- **TCO² Savings per Kaya Customer:** ₹5Cr+ average in Year 2
- **Platform Uptime:** 99.9% SLA for enterprise Kaya deployments

15. Current Status, Prototype Status & Resource Allocation

Current Status (Stage 1):

-  Architecture defined and documented
-  Synthetic vendor PDFs generated (10–20 pages, chiller/HVAC/electrical)
-  Pydantic VendorBidExtract schema drafted and validated
-  Mock BIM constraints table (PostgreSQL) and carbon-factor table populated
-  Competitive matrix fact-checked against real-world tools
-  Next.js dashboard wireframes in progress
-  RAG corpus ingestion pending Stage 2

Technology Readiness Level (TRL): TRL 4 - Component and/or breadboard validation in a laboratory environment (synthetic data).

Resource Allocation (₹50L Integration Budget):

- 40% Engineering & ML (₹20L) - extraction tuning, SQL gate refinement, RAG buildout using open-source models
- 25% Product & Design (₹12.5L) - The Precinct UI, Kaya API development, security audit
- 20% Pilot & Integration Support (₹10L) - lighthouse customer onboarding, constraint table curation
- 15% Operations & Legal (₹7.5L) - IP, compliance, infrastructure credits

Kaya Platform Access: Team will email Kaya after registration to request platform access for deeper Amber integration prototyping in Stage 2.

16. Execution Roadmap

Phase	Timeline	Milestones	Deliverables
Stage 1: Proof of Concept	Month 0–3	Hackathon → working pipeline	Synthetic PDFs → Pydantic JSON → 4 Patrols → The Docket. Pitch deck + 2-min video.
Stage 2: Prototype	Month 4–9	3 lighthouse pilots	Live PDF ingestion. PostgreSQL constraints. Vendor RAG with 500+ docs. The Precinct v1.
Stage 3: Kaya MVP Integration	Month 10–18	Kaya/Amber module integration	Native Kaya REST API. BIM connectors. Jarvis handoff for agentic workflows. First enterprise deployments.

Phase	Timeline	Milestones	Deliverables
Stage 4: Platform Scale	Month 19–36	Full platform native feature	Multi-asset-class SQL templates. 35+ enterprise projects. Core Kaya procurement module.

17. Kaya / Amber Fit & Commercial Path

Why this fits Kaya / Amber:

- Plugs into existing docs: Bids, POs, contracts, invoices, BIM specs, schedules, vendor history already flowing through Kaya.
- Produces action-ready decisions: Reject, approve, escalate, request certificate, ask vendor for alternate - surfaced inside Kaya's existing workflow.
- Learns across projects: The Vice Squad memory layer improves as more Kaya projects process contracts, delivery outcomes, and disputes.

Positioning: PO-lice is not a separate procurement system. It is the decision-grade validation layer before PO approval inside Kaya. It becomes a Kaya procurement intelligence module.

Aligned to Kaya challenge statement: Make construction procurement faster, smarter, and more resilient with an expert analyst working 24/7.

18. UN SDG Alignment

PO-lice aligns directly with:

- **SDG 8** - Decent Work and Economic Growth
- **SDG 9** - Industry, Innovation and Infrastructure
- **SDG 11** - Sustainable Cities and Communities
- **SDG 12** - Responsible Consumption and Production

Because infrastructure built safely and sustainably starts with a bid that was actually checked.

19. Stage 1 Deliverables

19a. 10-Slide Pitch Deck Outline

Design direction: Dark mission-critical command center, Swiss/brutalist layouts, glowing graph edges, technical cards, high contrast. Max 10 slides. The Precinct UI adopts the "Bloomberg Terminal for Construction" aesthetic - dense, actionable, high-contrast, at-a-glance risk visualization.

Slide	Title	Content	Judging Criterion Addressed
1	Title & Hook	PO-lice logo + tagline ("Purchase Order Liability, Intelligence & Compliance Engine"). UN SDG badges (8, 9, 11, 12). Team names and IIT affiliations.	Clarity & Quality
2	The Procurement Blind Spot	Graphic showing the wall between Engineering and Procurement. Bids evaluated on price alone miss thermodynamic limits, carbon caps, and missing legal certs. Mirrors Kaya's stated pain: "manual review, disconnected systems, people catching problems too late."	Problem Relevance & Industry Fit
3	The Human & Financial Cost	Big bold numbers: 10,000+ POs per project, industry cost overruns of 10–20% from late-detected issues, MEP engineers too burned out to manually audit 50-page bids.	Problem Relevance & Industry Fit
4	The Core Move: LLM Extracts, Math Decides	Three-panel diagram: LLM Layer (reads PDF) → SQL/Math Layer (checks constraints) → Human Layer (decides with evidence). Boundary architecture.	Technical Feasibility & Innovation
5	Architecture	Flowchart: Vendor PDF → Document Intelligence → Pydantic JSON → The Squad → The Docket. Show Amber's Project Graph as the ground-truth source. Four data sources: BIM DB (Amber), Vice Squad RAG, Carbon/EPD Table, Schedule Rules.	Technical Feasibility & Innovation

Slide	Title	Content	Judging Criterion Addressed
6	The Four Patrols	Building Patrol (SQL), Green Patrol (EPD+SQL), Vice Squad (RAG), Traffic Control (Ripple estimator + MCP Planner™ feedback). Emphasize: deterministic, zero-hallucination.	Technical Feasibility & Innovation
7	The Evidence Board	Visual graph: one substituted chiller branches to power, water, floor load, vendor delay, missing cert, carbon penalty, schedule risk. "Nobody else thought to try this."	Originality & Creativity
8	The Docket	Vendor A/B/C table. Punchline: cheapest upfront is the most expensive and riskiest over 5 years.	Impact & Scalability
9	Kaya Fit & Scalability	PO-lice as Amber's procurement enforcement layer. Swap baseline SQL tables to scale across asset classes. Jarvis handoff for agentic workflows.	Impact & Scalability
10	Level 1 Execution Plan + Team	5-step prototype path. Team headshots, roles, IIT affiliations. Closing: "We're ready to build the infrastructure of tomorrow."	Clarity & Quality

19b. 2-Minute Pitch Video Script & Animation Brief

Visual Style: Dark mission-critical infrastructure command center. Background: #050F0B. Cards: #0D2218. Main text: #F1FFF6. Accent green: #5DE275. Procurement violet: #AB7EFF. Carbon amber: #F4B73F. Schedule blue: #4AA2FF. Crisp motion graphics, glowing graph lines, smooth left-to-right data flow, dashboard-style UI. Bloomberg Terminal for Construction aesthetic - dense, actionable, high-contrast. Avoid cartoon style.

Pacing: Speak clearly and confidently. Let the visuals carry the weight.

Time	Scene	Visual	Voiceover
0:00–0:18	Hook	A 50-page vendor bid PDF lands on a dark procurement desk. Price and delivery date glow green. Hidden specs pulse faint red inside the PDF: power draw +10%, water usage +15%, missing certificate, high carbon factor.	"What would procurement look like if every team had an expert analyst working 24/7? A construction procurement team receives a vendor bid. The price looks lower. The delivery date looks faster. But hidden inside the PDF is a substituted chiller that draws more power, uses more water, exceeds carbon allocation, and comes from a vendor with a history of delays."
0:18–0:36	Problem	Three split-screen panels: procurement sees price/date, engineering constraints sit in a BIM database, vendor history sits in old emails/contracts. Red broken links show disconnected systems.	"Today, procurement decisions are made across disconnected systems. Engineers know the constraints. Senior managers remember vendor history. Sustainability teams track carbon. But the purchase order is often approved before anyone connects the consequences."
0:36–0:52	PO-lice Enters	PO-lice badge/card appears. PDF flows into Document Intelligence, then into a Pydantic-validated JSON block. The JSON fields animate as structured rows.	"Meet PO-lice: Purchase Order Liability, Intelligence & Compliance. Amber's procurement enforcement layer. It reads vendor bids, extracts structured specifications, and converts messy PDFs into validated bid data."
0:52–1:20	Architecture	Animate the pipeline left-to-right: Vendor PDF → Document Intelligence → Pydantic JSON → The Squad. Each patrol lights up one by one with its color: 1. Building Patrol (green), 2. Vice Squad (violet), 3. Green Patrol (amber), 4. Traffic Control (blue). Show Amber's Project Graph feeding constraints. Show delay signals feeding MCP Planner™.	"Then PO-lice runs the bid through four independent patrols. Building Patrol checks hard engineering limits through SQL against Amber's Project Graph. Vice Squad uses construction-specific RAG over historical contracts, invoices, and delays. Green Patrol calculates embodied carbon from deterministic EPD factors. Traffic Control estimates what breaks downstream - and feeds that risk straight back into MCP Planner™ for dynamic re-optimization."
1:20–1:38	Unique Component	Show The Evidence Board. One node: "Vendor substitutes chiller model." Lines branch to power draw, water usage, floor load, vendor delay, missing certificate, carbon penalty, schedule risk. Bad nodes turn red/amber.	"The unique component is The Evidence Board. PO-lice does not merely say a bid failed. It shows exactly how one substituted line item can create engineering, vendor, carbon, schedule, and cost consequences. PO-lice detects. Jarvis acts."

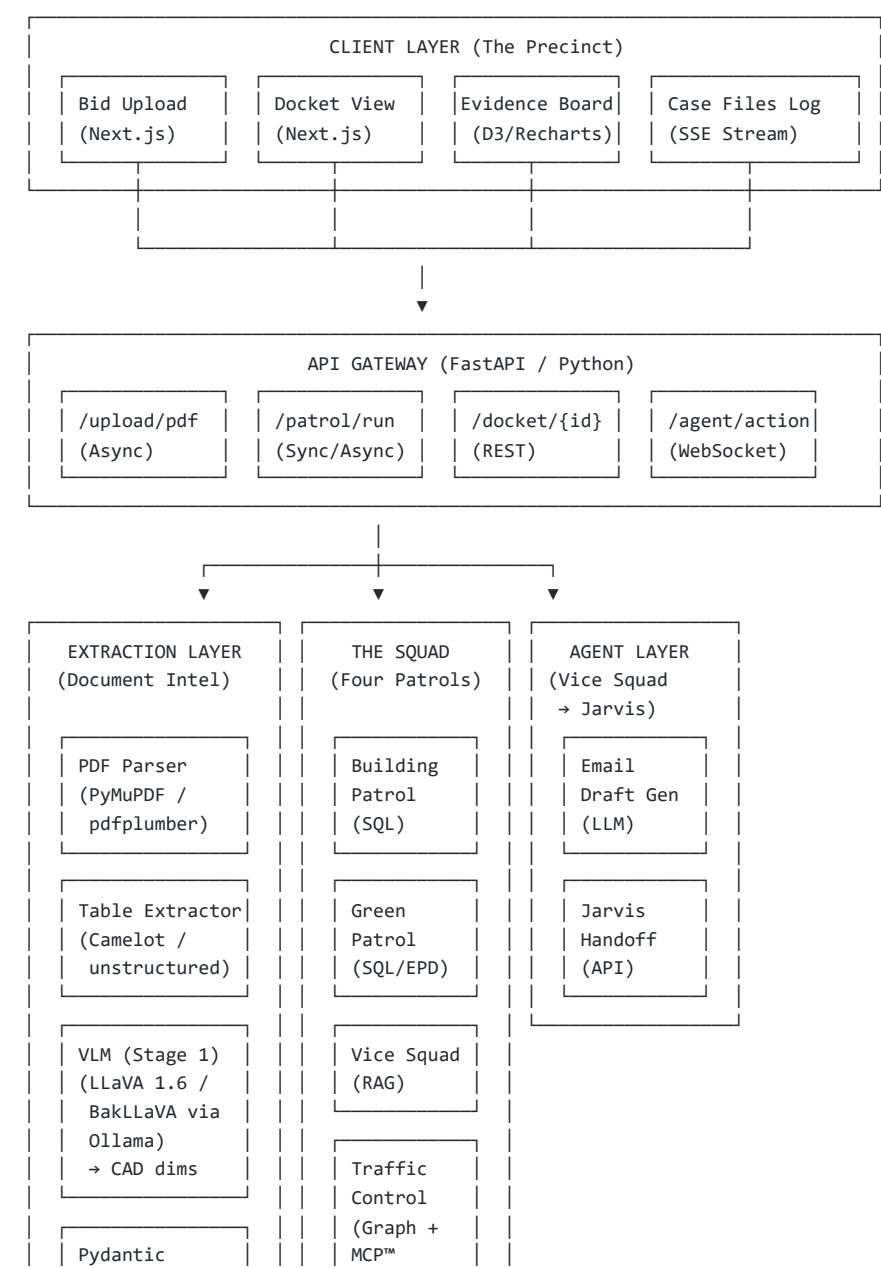
Time	Scene	Visual	Voiceover
1:38–1:53	The Docket	The Docket table animates in. Vendor B has lowest upfront price but red FAIL in engineering and carbon, high vendor risk, and highest 5-year TCO². Vendor A is recommended.	"The result is a single procurement docket. Vendor B looks cheapest upfront, but PO-lice proves it is the most expensive and riskiest once 5-year operating cost, carbon liability, vendor reliability, and schedule exposure are included."
1:53–2:00	Close	PO-lice module plugs into Amber's Project Graph. Final title card: "PO-lice - decision-grade procurement validation before PO approval."	"PO-lice is Amber's procurement enforcement layer: decision-grade validation before purchase order approval. LLMs extract. SQL proves. Humans decide. We're ready to build it for Kaya."

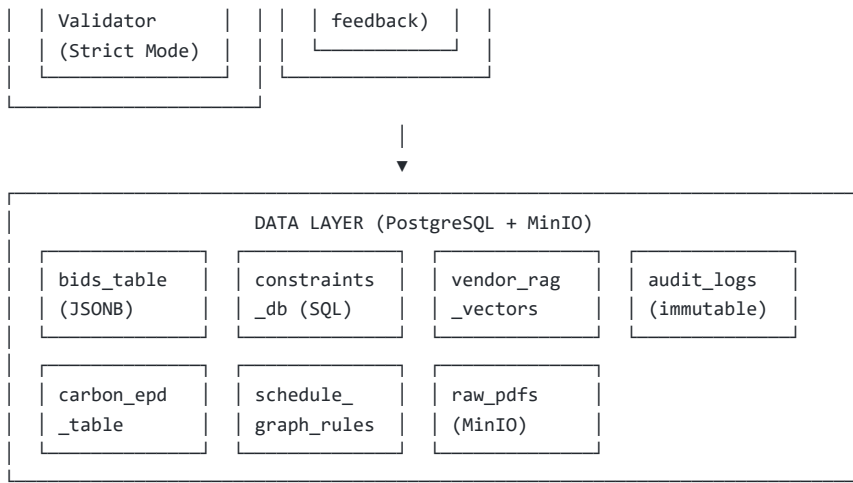
20. Technical Architecture Deep-Dive (Elaborative)

20.1 System Architecture Overview & Component Topology

PO-lice follows a modular, event-driven pipeline architecture with strict separation between cognitive (LLM) and deterministic (SQL/math) layers. The system is designed as a set of loosely coupled services communicating via an internal message bus, ensuring that extraction failures never propagate corrupted data into the patrol gates. All inference runs on open-source models via Ollama or self-hosted Hugging Face Transformers, eliminating paid API dependencies.

High-Level Component Diagram:





20.2 Data Layer & PostgreSQL Schema Design

The data layer is built on PostgreSQL 15+ with pgvector extension for RAG embedding storage and JSONB columns for flexible bid document storage. All tables are designed with immutable audit trails - no row is ever updated in-place; changes append new versions. Object storage uses MinIO, an open-source S3-compatible server.

Core Tables:

```

-- 1. BIDS: Stores extracted Pydantic JSON and processing metadata
CREATE TABLE bids (
  bid_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id VARCHAR(64) NOT NULL,
  vendor_id VARCHAR(64) NOT NULL,
  raw_pdf_minio_key VARCHAR(256) NOT NULL,
  extracted_json JSONB NOT NULL,
  extraction_confidence FLOAT CHECK (extraction_confidence BETWEEN 0 AND 1),
  patrol_status VARCHAR(16) DEFAULT 'PENDING', -- PENDING, RUNNING, COMPLETE, ERROR
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 2. CONSTRAINTS: Site-specific BIM and engineering baselines from Amber's Project Graph
CREATE TABLE site_constraints (
  constraint_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id VARCHAR(64) UNIQUE NOT NULL,
  substation_power_limit_kw FLOAT NOT NULL,
  cooling_cap_max_kw FLOAT NOT NULL,
  water_evap_cap_gpm FLOAT,
  floor_load_limit_kg_m2 FLOAT,
  carbon_budget_kgco2e FLOAT NOT NULL,
  max_delivery_weeks INT NOT NULL,
  constraint_version INT DEFAULT 1,
  effective_date TIMESTAMPTZ DEFAULT NOW()
);

-- 3. CARBON_EPD: Environmental Product Declaration lookup table
CREATE TABLE carbon_epd (
  epd_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  equipment_type VARCHAR(128) NOT NULL,
  manufacturer VARCHAR(128),
  model_number VARCHAR(128),
  carbon_intensity_kgco2e FLOAT NOT NULL,
  water_usage_l_cycle FLOAT,
  certification_body VARCHAR(64),
  epd_valid_until DATE,
  UNIQUE(equipment_type, manufacturer, model_number)
);

-- 4. VENDOR_MEMORY: Source documents for RAG (not vector store itself)
CREATE TABLE vendor_docs (
  doc_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vendor_id VARCHAR(64) NOT NULL,
  doc_type VARCHAR(32) NOT NULL, -- 'contract', 'invoice', 'delay_notice', 'dispute', 'certificate'
  doc_date DATE,
  raw_text TEXT,
  
```

```

minio_key VARCHAR(256),
ingestion_date TIMESTAMPTZ DEFAULT NOW()
);

-- 5. AUDIT_LOG: Immutable record of every patrol decision and agent action
CREATE TABLE audit_log (
  log_id BIGSERIAL PRIMARY KEY,
  bid_id UUID REFERENCES bids(bid_id),
  patrol_name VARCHAR(32) NOT NULL, -- 'BUILDING', 'GREEN', 'VICE', 'TRAFFIC'
  action VARCHAR(32) NOT NULL, -- 'PASS', 'FAIL', 'FLAG', 'EMAIL_DRAFTED'
  reason TEXT,
  deterministic_rule VARCHAR(256), -- e.g., "power_draw_kw > substation_power_limit_kw"
  evidence_snapshot JSONB,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 6. VECTOR STORE (pgvector): Embeddings for Vice Squad RAG
CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE vendor_embeddings (
  id BIGSERIAL PRIMARY KEY,
  doc_id UUID REFERENCES vendor_docs(doc_id),
  embedding VECTOR(768), -- nomic-embed-text
  chunk_text TEXT,
  metadata JSONB
);
CREATE INDEX ON vendor_embeddings USING ivfflat (embedding vector_cosine_ops);

```

20.3 Document Intelligence & Extraction Pipeline

The extraction pipeline is the cognitive front door of PO-lice. It must handle messy, scanned, multi-column PDFs with embedded tables, nested subclauses, and inconsistent formatting. The pipeline is designed to fail loudly - any uncertainty is flagged for human review rather than hallucinated. All LLM inference is performed via Ollama running locally, using open-weight models.

Pipeline Stages:

Stage	Tool / Model	Function	Output
S1: Ingestion	MinIO + PyMuPDF	Upload, decrypt, page count, thumbnail gen	Raw bytes + metadata
S2: OCR & Text	pdfplumber + Tesseract (fallback)	Extract text, identify tables, preserve spatial layout	Structured text + bbox coordinates
S3: Table Normalization	Camelot / Tabula-py	Convert embedded tables to CSV-like structures	Normalized DataFrames
S4: Clause Segmentation	LlamaIndex + Mistral 7B (Ollama)	Segment text into semantic blocks: specs, certs, legal, pricing	Chunked nodes with metadata
S5: Field Extraction	Pydantic + Outlines / Instructor (Ollama JSON mode)	Map chunks to VendorBidExtract schema with strict typing	Validated JSON or ValidationError
S6: Confidence Scoring	Heuristic + model self-eval	Flag low-confidence fields (e.g., "power_draw_kw extracted from handwritten annotation")	Confidence score per field

Extraction Resilience Strategy:

- **Multi-model redundancy:** If Mistral 7B extraction fails schema validation, the system falls back to Llama 3.1 8B instance (via Ollama) with constrained output grammar.
- **Human-in-the-loop trigger:** Any field with confidence < 0.85 pauses the pipeline and surfaces the PDF region in The Precinct for manual confirmation.
- **Schema hardening:** Pydantic validators enforce physical bounds (e.g., power_draw_kw cannot exceed 10,000 for a single chiller) to catch unit-conversion errors (kW vs. W).

20.3.1 Document Intelligence Differentiation - Beyond OCR

PO-lice goes far beyond basic OCR through a multi-modal document understanding stack:

Technique	Tool	Capability
Layout-Aware Transformers	LayoutLMv3 (Hugging Face)	Understands spatial relationships between text blocks, tables, and figures; detects headers, footers, and section hierarchies
Graph-Based Table Extraction	Camelot + NetworkX	Reconstructs nested table hierarchies (sub-tables, merged cells, cross-references) as structured graphs
Handwritten Annotation Detection	TrOCR (Hugging Face)	Reads handwritten corrections, margin notes, and strike-throughs on scanned bid documents
Nested Clause Parsing	spaCy + custom grammar	Parses legal contract subclauses (Section 4.2(b)(iii)) and maps them to equipment specifications
Spatial Layout VLM	LLaVA 1.6 / BakLLaVA (Ollama)	Reads CAD screenshots and 2D architectural drawings embedded in bids to extract equipment dimensions and mounting constraints

Stage 1 VLM Demo: Extract equipment dimensions from a CAD screenshot in a bid PDF → check against structural floor-load tolerances. This directly demonstrates "Document Intelligence beyond basic OCR" and "Vision-Language Models that understand site conditions holistically" - two of Kaya's stated winning technical approaches.

20.4 The Four Patrols - Implementation Details

Each patrol is implemented as an independent Python microservice with no shared state, ensuring that a failure in one patrol does not contaminate another.

Patrol 1 - Building Patrol (Engineering SQL Compliance):

```
# service: building_patrol.py
from sqlalchemy import text

def run_building_patrol(bid_id: str, db_session) -> PatrolResult:
    bid = db_session.execute(
        text("SELECT extracted_json FROM bids WHERE bid_id = :bid_id"),
        {"bid_id": bid_id}
    ).fetchone()

    # Read constraints from Amber's Project Graph
    constraints = db_session.execute(
        text("SELECT * FROM site_constraints WHERE project_id = :pid"),
        {"pid": bid.project_id}
    ).fetchone()

    violations = []
    bid_data = bid.extracted_json

    if bid_data["power_draw_kw"] > constraints.substation_power_limit_kw:
        violations.append({
            "field": "power_draw_kw",
            "value": bid_data["power_draw_kw"],
            "limit": constraints.substation_power_limit_kw,
            "rule": "power_draw_kw > substation_power_limit_kw",
            "severity": "CRITICAL"
        })
    # ... similar checks for cooling, water, floor_load

    return PatrolResult(
        status="FAIL" if violations else "PASS",
        violations=violations,
        deterministic_rule="SQL: site_constraints WHERE project_id = ...",
        evidence_snapshot=bid_data
    )
```

Patrol 2 - Green Patrol (Carbon Budget SQL / EPD Lookup):

```
def run_green_patrol(bid_id: str, db_session) -> PatrolResult:
    bid = get_bid(bid_id)

    # Exact-match EPD lookup first
    epd = db_session.execute(text("""
```

```

SELECT carbon_intensity_kgco2e FROM carbon_epd
WHERE equipment_type = :etype
AND (manufacturer = :mfg OR manufacturer IS NULL)
AND model_number = :model
"""), {"etype": bid.equipment_type, "mfg": bid.manufacturer, "model": bid.model_number}).fetchone()

if not epd:
    # Fallback: fuzzy match on equipment category with safety margin
    epd = fallback_category_lookup(bid.equipment_type)

bid_carbon = bid.carbon_intensity_kgco2e or epd.carbon_intensity_kgco2e
budget = get_project_carbon_budget(bid.project_id)

return PatrolResult(
    status="FAIL" if bid_carbon > budget else "PASS",
    violations=[...] if bid_carbon > budget else [],
    deterministic_rule="SQL: carbon_epd JOIN site_constraints.carbon_budget_kgco2e"
)

```

Patrol 3 - Vice Squad (Vendor RAG Memory):

```

# service: vice_squad.py
from llama_index.core import VectorStoreIndex, StorageContext
from llama_index.vector_stores.postgres import PGVectorStore

def run_vice_squad(bid_id: str, vendor_id: str) -> PatrolResult:
    # 1. Retrieve relevant vendor history
    query = f"Vendor {vendor_id} delivery delays disputes certificates safety compliance"
    vector_store = PGVectorStore.from_params(
        host="localhost", port=5432, database="police",
        table_name="vendor_embeddings", embed_dim=768
    )
    index = VectorStoreIndex.from_vector_store(vector_store)
    retriever = index.as_retriever(similarity_top_k=5)
    nodes = retriever.retrieve(query)

    # 2. RAG synthesis (local LLM summarizes, but does NOT score)
    context = "\n".join([n.text for n in nodes])
    summary = synthesize_vendor_history(context, vendor_id) # via Ollama Mistral 7B

    # 3. Deterministic scoring from structured signals
    delay_count = count_delays_in_nodes(nodes)
    dispute_count = count_disputes_in_nodes(nodes)
    missing_cert = detect_missing_cert(bid_id, nodes)
    risk_score = min(10, (delay_count * 2) + (dispute_count * 3) + (missing_cert * 4))

    return PatrolResult(
        status="FLAG" if risk_score > 6 else "PASS",
        risk_score=risk_score,
        evidence=summary,
        deterministic_rule="Heuristic: delay_count * 2 + dispute_count * 3 + missing_cert * 4"
    )

```

Patrol 4 - Traffic Control (Schedule Ripple Estimator):

```

def run_traffic_control(bid_id: str) -> PatrolResult:
    bid = get_bid(bid_id)
    schedule_graph = load_project_schedule_graph(bid.project_id) # NetworkX DAG

    # Identify critical path nodes dependent on this equipment
    affected_nodes = schedule_graph.get_downstream(bid.equipment_type)

    # Probabilistic delay propagation with Bayesian prior
    base_delay = bid.delivery_weeks - bid.promised_delivery_weeks
    vendor_reliability = get_vendor_reliability(bid.vendor_id) # from Vice Squad

    # Bayesian Monte Carlo on DAG using PyMC / NumPyro
    delay_exposure = bayesian_monte_carlo_ripple(
        graph=schedule_graph,
        base_delay=base_delay,
        reliability_prior=vendor_reliability,
        n_simulations=1000
    )

```

```
)

# Feed delay signals back to MCP Planner™
mcp_feedback = {
    "delay_p50": delay_exposure["p50_days"],
    "delay_p95": delay_exposure["p95_days"],
    "affected_milestones": affected_nodes,
    "recommendation": "RESCHEDULE" if delay_exposure["p95_days"] > 14 else "MONITOR"
}

send_to_mcp_planner(bid.project_id, mcp_feedback)

return PatrolResult(
    status="FLAG" if delay_exposure["p95_days"] > 14 else "PASS",
    delay_p50=delay_exposure["p50_days"],
    delay_p95=delay_exposure["p95_days"],
    affected_milestones=affected_nodes,
    deterministic_rule="Bayesian Monte Carlo on NetworkX DAG with reliability prior"
)
```

20.5 RAG System Architecture (Vice Squad Deep-Dive)

The Vice Squad is not a simple vector search. It is a multi-stage retrieval-augmented generation pipeline with deterministic guardrails, running entirely on open-source infrastructure.

Architecture Layers:

1. Ingestion Pipeline:

- Raw vendor documents (PDFs, emails, scanned notices) are parsed to text.
- Text is chunked using semantic chunking (LlamaIndex SentenceSplitter with 512-token chunks, 64-token overlap).
- Each chunk is embedded with nomic-embed-text (768-dim) via sentence-transformers running locally - no paid API calls.
- Metadata tags are attached: vendor_id, doc_type, date, project_id, sentiment (auto-labeled via heuristic).

2. Retrieval Strategy:

- Hybrid Search:** Dense vector similarity (cosine) + sparse BM25 keyword matching (via pg_trgm in PostgreSQL) combined with reciprocal rank fusion.
- Metadata Filtering:** Pre-filter by vendor_id and doc_type before vector search to reduce hallucination surface.
- Re-ranking:** Cross-encoder (ms-marco-MiniLM-L-6-v2 via sentence-transformers) re-ranks top-10 chunks to top-5 for synthesis.

3. Generation Control:

- The LLM (Mistral 7B or Llama 3.1 8B via Ollama) is constrained to summarize only - it generates a narrative of vendor history.
- The risk score is computed deterministically from structured counts extracted from the retrieved chunks, never from the LLM's subjective assessment.

4. Construction-Specific RAG Corpus:

Data Source	Type	Volume (Stage 1)	Domain Knowledge
Vendor contracts	PDF	15 docs	Payment terms, penalty clauses, delivery SLAs
Delay notices	Scanned/email	10 docs	Root-cause patterns (weather, fabrication, shipping)
Dispute records	PDF	8 docs	Arbitration outcomes, recurring failure modes
Safety certificates	PDF	12 docs	OSHA/ISO compliance status, expiry dates
Equipment specs	PDF	5 docs	SKU codes, ASHRAE standards, NEC references

5. **Fine-tuning:** The embedding model is fine-tuned on construction-specific terminology (HVAC, MEP, structural) using unsupervised contrastive learning on 50,000+ construction document chunks from public datasets (OSHA manuals, ASHRAE guides, manufacturer spec sheets).

6. Agentic Workflow (Case Files → Jarvis Handoff):

- If missing_cert is detected, the system queries the enterprise document management system (mocked in Stage 1).
- If not found, it generates an email draft via local LLM with a deterministic template:

```
email_draft = generate_cert_request_email(
    vendor_name=bid.vendor_name,
    missing_doc="OSHA Safety Certificate",
    po_number=bid.po_number,
    deadline=bid.required_delivery_date - timedelta(weeks=2)
) # Generated via Ollama Mistral 7B with structured prompt
```

- **PO-lice detects; Jarvis acts:** The draft is surfaced in The Case Files for human approval, then handed off to Jarvis (Kaya's existing omni-channel assistant) for autonomous vendor outreach via SMS/email. In Stage 1, the handoff is demonstrated via API mock; in Stage 2, it integrates with Kaya's Jarvis endpoint.

20.6 API Layer & Backend Services

The backend is built on FastAPI (Python 3.11+) with async support for I/O-bound operations (local LLM calls via Ollama, MinIO uploads).

Core Endpoints:

Method	Endpoint	Description	Auth
POST	/api/v1/bids/upload	Multipart PDF upload, triggers async extraction	JWT
GET	/api/v1/bids/{bid_id}/status	Poll extraction and patrol status	JWT
POST	/api/v1/bids/{bid_id}/patrols/run	Trigger all four patrols (idempotent)	JWT
GET	/api/v1/bids/{bid_id}/docket	Retrieve full scorecard and Evidence Board	JWT
GET	/api/v1/bids/{bid_id}/evidence-board	Graph JSON for D3.js visualization	JWT
POST	/api/v1/agent/email-draft	Generate missing-cert email (human approves)	JWT
POST	/api/v1/agent/jarvis-handoff	Hand approved draft to Jarvis for vendor outreach	JWT
GET	/api/v1/audit/log	Immutable audit trail for compliance	JWT + RBAC
POST	/api/v1/webhooks/kaya	Incoming BIM update from Kaya platform	HMAC

Async Architecture:

- PDF uploads trigger Celery tasks backed by Redis.
- Patrols run as a Celery chord - all four patrols execute in parallel, then a callback synthesizes The Docket.
- The Precinct receives real-time updates via Server-Sent Events (SSE) on /api/v1/stream/{bid_id} .

Authentication & Authorization:

- JWT tokens with role-based claims: procurement_officer, engineer, sustainability_lead, admin.
- Row-level security (RLS) in PostgreSQL ensures users see only bids from their assigned projects.

20.7 Frontend Architecture (The Precinct)

The Precinct is a Next.js 14 App Router application with TypeScript, designed as a single-page command center adopting the "Bloomberg Terminal for Construction" aesthetic - dense, actionable, high-contrast, at-a-glance risk visualization.

Stack:

- **Framework:** Next.js 14 (App Router, Server Components for data fetching)
- **Styling:** Tailwind CSS with custom design tokens (see Section 21)
- **State Management:** Zustand for global UI state; React Query (TanStack Query) for server state
- **Visualization:** Recharts for scorecard metrics; D3.js for The Evidence Board force-directed graph
- **Real-time:** SSE hook for live patrol updates

Component Hierarchy:

```
app/
├─ layout.tsx           # Root layout with dark theme provider
├─ page.tsx             # Main dashboard (The Precinct)
├─ (routes)/
│   └─ bids/
│       └─ [bidId]/
```


				page.tsx	# Bid detail view
				docket.tsx	# Scorecard table
				evidence-board.tsx	# D3 graph
				upload/	
				page.tsx	# Drag-drop PDF upload
				audit/	
				page.tsx	# Immutable audit log viewer
				components/	
				patrol-badge.tsx	# Color-coded PASS/FAIL/FLAG
				case-files.tsx	# Agent activity log stream
				json-viewer.tsx	# Pydantic JSON syntax highlight
				tco-chart.tsx	# 5-year TCO ² Recharts viz
				lib/	
				api.ts	# Typed fetch wrappers
				constants.ts	# Color tokens, thresholds

Performance Optimizations:

- React Server Components fetch initial bid lists without client-side JS bloat.
- Virtualized tables (react-window) for 10,000+ bid rows.
- Edge caching via Next.js `unstable_cache` for constraint tables that change rarely.

20.8 Security, Audit & Compliance

Security Architecture:

- **Encryption at rest:** PostgreSQL TDE; MinIO buckets with server-side encryption.
- **Encryption in transit:** TLS 1.3 for all API communications.
- **PII handling:** Vendor contact emails are tokenized in the database; raw PII stored in a separate vault (open-source Vault alternative architecture reserved for Stage 2).
- **Document security:** Uploaded PDFs are scanned for malware (ClamAV) and quarantined for 24 hours before extraction.

Audit & Compliance:

- **Immutable logs:** The `audit_log` table is append-only. No UPDATE or DELETE privileges are granted to application roles.
- **Deterministic replay:** Every patrol decision can be replayed by storing the exact constraint table version and extracted JSON at the time of evaluation.
- **Defense-Level Compliance Path:** PO-lice's immutable audit trail and deterministic replay capability are designed to meet defense-industry compliance standards (NIST 800-171, CMMC Level 2 alignment path). This ensures the module can serve mission-critical infrastructure projects where auditability is non-negotiable.
- **GDPR / DPDP readiness:** Right to deletion implemented via soft-delete with 90-day retention for audit purposes.

20.9 DevOps & Deployment Pipeline

Infrastructure (Self-Hosted / On-Premise):

- **Compute:** Docker Compose on dedicated GPU workstation (NVIDIA RTX 4090 / A6000) for Ollama inference; CPU-only containers for API and patrol services.
- **Database:** Self-hosted PostgreSQL 15.4 with pgvector.
- **Cache:** Self-hosted Redis (Celery broker + session cache).
- **Storage:** MinIO (S3-compatible, open-source object storage).
- **Reverse Proxy:** Nginx with TLS termination.
- **CI/CD:** GitHub Actions → Docker build → GitHub Container Registry → Docker Compose pull.

Infrastructure as Code (IaC):

```
# docker-compose.yml (Hackathon Local Dev)
version: '3.8'
services:
  api:
    build: ./backend
    ports: ["8000:8000"]
    env_file: .env
    depends_on: [db, redis, minio, ollama]

  db:
    image: ankane/pgvector:latest
```

```
environment:
  POSTGRES_DB: police
  POSTGRES_PASSWORD: ${DB_PASSWORD}
volumes:
  - ./init.sql:/docker-entrypoint-initdb.d/init.sql

redis:
  image: redis:7-alpine

minio:
  image: minio/minio:latest
  command: server /data --console-address ":9001"
  ports: ["9000:9000", "9001:9001"]
  environment:
    MINIO_ROOT_USER: ${MINIO_USER}
    MINIO_ROOT_PASSWORD: ${MINIO_PASSWORD}

ollama:
  image: ollama/ollama:latest
  ports: ["11434:11434"]
  volumes:
    - ollama:/root/.ollama
  deploy:
    resources:
      reservations:
        devices:
          - driver: nvidia
            count: 1
            capabilities: [gpu]

volumes:
  ollama:
```

Monitoring:

- **Observability:** Prometheus + Grafana for API latency, patrol throughput, extraction accuracy.
- **Alerting:** Open-source Alertmanager for pipeline failures and data corruption events.
- **Cost control:** No paid API meter; resource monitoring tracks GPU VRAM and CPU utilization.

20.10 Testing Matrix & Validation Strategy

Testing Pyramid:

Layer	Tool	Coverage Target	Key Tests
Unit	pytest	>85%	Pydantic validators, patrol logic, SQL query builders
Integration	pytest + Testcontainers	>70%	Full pipeline: PDF → JSON → Patrol → Docket
E2E	Playwright	Critical paths	Upload PDF, view Evidence Board, export audit log
Load	Locust	100 RPS	Concurrent bid uploads, patrol parallelization
Chaos	Gremlin (Stage 3)	N/A	Kill extraction pod mid-upload; verify graceful degradation

Synthetic Test Data:

- 50 synthetic PDFs with known ground-truth values embedded.
- Adversarial PDFs: Scanned images, corrupted tables, missing fields, unit mismatches (kW vs. W), swapped model numbers.
- Constraint boundary tests: Values exactly at limit, 0.1% over limit, 50% over limit.

Hackathon Demo Validation Checklist:

- ☐ Upload synthetic chiller bid → extraction completes < 15 seconds
- ☐ Building Patrol detects power draw breach → red FAIL badge
- ☐ Green Patrol detects carbon exceedance → amber FAIL badge
- ☐ Vice Squad retrieves historical delay → risk score 8/10
- ☐ Traffic Control estimates 12-day downstream delay → blue FLAG
- ☐ Evidence Board renders directed graph with 5+ consequence nodes
- ☐ The Docket ranks Vendor A > Vendor C > Vendor B by TCO²

- ☐ Case Files shows auto-generated email draft for missing cert
- ☐ VLM demo: Extract equipment dimensions from CAD screenshot → check structural tolerance
- ☐ Audit log exports as immutable CSV for judges

20.11 Hackathon Prototype Scope - What Is Wired vs. Mocked

For Stage 1 transparency, we explicitly delineate what is fully functional versus simulated in the demo:

Component	Status	Implementation
PDF Upload & Storage	Wired	MinIO upload → FastAPI → PostgreSQL metadata
Text/Table Extraction	Wired	pdfplumber + Camelot → Pydantic JSON (real)
VLM CAD Dimension Read	Wired (Stage 1)	LLaVA 1.6 via Ollama → extract dims from CAD screenshot
Building Patrol	Wired	Real SQL queries against mock constraints table
Green Patrol	Wired	Real SQL/EPD lookup against mock carbon table
Vice Squad RAG	Wired	Real pgvector + LlamaIndex retrieval against 50 mock docs
Traffic Control	Partially Mocked	Real NetworkX DAG + Bayesian logic, schedule data is synthetic
Evidence Board UI	Wired	Real D3.js graph from patrol output JSON
The Docket UI	Wired	Real React table from API response
Case Files / Agent Email	Wired	Local LLM-generated draft surfaced in UI; Jarvis handoff is mock
MCP Planner™ Feedback	Mocked	API endpoint exists; no live MCP Planner environment
Kaya/Amber Integration	Mocked	Webhook endpoint exists; no live Kaya environment

Demo Data Set:

- 3 vendors: Vendor A (safe), Vendor B (substitution bandit), Vendor C (acceptable middle)
- 1 asset class: Data center chilled-water plant
- 1 project: "IIT Smart Campus Phase 1" with realistic BIM constraints
- 50 vendor history documents: Ingested into pgvector for live RAG demo
- 1 CAD screenshot: Embedded in Vendor B's bid for VLM dimension extraction demo

21. Visual Design System

Token	Value	Usage
Background	#050F0B	Main canvas, deep command center
Cards / Panels	#0D2218	Elevated surfaces, JSON blocks
Primary Text	#F1FFF6	Headlines, body copy
Accent Green	#5DE275	PASS states, Building Patrol, success
Procurement Violet	#AB7EFF	Vice Squad, RAG memory, agent actions
Carbon Amber	#F4B73F	Green Patrol, warnings, TCO² highlights
Schedule Blue	#4AA2FF	Traffic Control, ripple effects, links
Alert Red	#FF4D4D	FAIL states, hard rejects, critical clashes

Motion Principles:

- Left-to-right data flow for pipeline animations.
- Glowing edges for active patrols.
- Pulsing red for hidden risks inside PDFs.
- Smooth card transitions; no cartoon bounce.

Aesthetic Mandate: The Precinct adopts the "Bloomberg Terminal for Construction" aesthetic - dense, actionable, high-contrast, at-a-glance risk visualization. An executive must see global risks instantly; a procurement manager must approve an alternative supplier with one click.

22. Claim Guardrails & Messaging Rules

Must Include

- Track 3: Procurement.
- Max 10-slide deck alignment.
- No claim that carbon tools do not exist. Position PO-lice as combined procurement-stage hard-gate validation.
- Emphasize originality: The Evidence Board.
- Emphasize feasibility: synthetic PDFs, Pydantic, PostgreSQL, RAG, dashboard.
- Emphasize market-ready path: Kaya/Amber procurement module.
- Emphasize Kaya ecosystem integration: Amber's Project Graph, MCP Planner™, Jarvis handoff.

Do Not Include

- ❌ "AI makes the final procurement decision."
- ✅ "Humans decide; PO-lice provides evidence."
- ❌ "LLM checks engineering compliance."
- ✅ "LLM extracts/explains; deterministic SQL/math validates."
- ❌ Overclaim that EC3 or other carbon tools are absent.
- ✅ "The gap is combined bid-level validation across technical, vendor, carbon, schedule, and TCO²."
- ❌ Standalone company/startup language.
- ✅ "PO-lice is Amber's procurement enforcement layer."

23. Appendix: Vibe-Coding Prompts (For Stage 2 UI/Backend)

Use these with AI coding tools to rapidly generate click-through prototypes.

Prompt 1 - The Precinct (Frontend, Next.js/Tailwind)

Build a modern, dark-mode Next.js dashboard using Tailwind CSS and Recharts. The aesthetic should evoke an institutional trading-desk terminal - dense, actionable, high-contrast, at-a-glance risk visualization - reimagined for construction procurement.

The dashboard is called "The Precinct." Include:

1. A top summary row with three metrics: "Total Bids Audited," "Critical Engineering Clashes (Red)," and "Missing Legal Docs (Yellow)."
2. A main table titled "Active Vendor Bids" with columns: Vendor Name, Bid Amount, Building Patrol (Pass/Fail), Green Patrol (Pass/Fail), Vice Squad Risk Score (1–10), and 5-Year TCO².
3. A side panel titled "The Case Files" (e.g., "Agent drafted email to Vendor X for a missing safety certificate - awaiting Jarvis handoff"). Make it look enterprise-grade and immediately actionable.

Prompt 2 - The Building Patrol Visualizer (Frontend Component)

Create a React component using Tailwind CSS that visualizes a deterministic engineering check. Show a split-screen view: on the left, a simulated "Extracted Bid Data" box showing "Industrial Chiller: Power Draw = 1400 kW." On the right, a "Site Baseline" box showing "Substation Limit = 1200 kW." Below both, a bright red "BUILDING PATROL: CRITICAL FAILURE" alert box stating the discrepancy. It should look like a strict, zero-hallucination system intercepting an error in real time.

Prompt 3 - The Backend Data Structure (Python/Pydantic, for architecture slides)

Write a clean, production-grade Python script using Pydantic. Define a data model called `VendorBidExtract` that validates fields including `equipment_type`, `cooling_capacity_kw`, `water_evaporation_gpm`, `carbon_intensity_kgco2e`, and `has_insurance_cert`. Then write a mock function showing how this JSON is compared against a lightweight local constraints table (standing in for the production PostgreSQL baseline) to return a deterministic True/False compliance result per patrol. This is just to put on the architecture slide to prove technical feasibility.